

Agentic Execution Boundary Failure Model

***Action Capability, Constraint Lag, and Irreversibility
in Tool-Using Systems***

Version: 0.1

Date: July 2026

Status: Preprint / OSF external-facing version

Context: Meta-Writing Ecology

Document Type: Conceptual Model

Huang Tzuyuan

Abstract

This document formalizes the **Agentic Execution Boundary Failure Model** as a structural framework for analyzing what changes when an interpretive or generative system acquires the capacity to execute actions through tools, services, accounts, devices, or institutional workflows. The model begins from a central distinction: a system that can recommend an action and a system that can perform the action do not occupy the same boundary condition.

Execution capability can expand faster than constraint architecture. A tool-using system may move from drafting, searching, and suggesting into sending, purchasing, modifying, deleting, publishing, transferring, or initiating operations. If permissions, verification, reversibility, scope, human review, and consequence monitoring remain designed for a non-executing system, the original boundary becomes insufficient. Low-friction action can then convert an interpretive error into an operational event before adequate review occurs.

The model identifies suggestion–action collapse, permission overhang, verification gaps, misplaced human-in-the-loop review, reversibility assumptions, and post-failure constraint residue. It is not an argument against agentic systems or automation. It provides a structural vocabulary for determining when action capacity has crossed a boundary without a corresponding redesign of authority, confirmation, rollback, traceability, and answerability.

The framework is applicable to digital agents, administrative automation, financial and procurement workflows, publication systems, infrastructure management, and other settings in which generated judgments can produce external effects.

Keywords

Agentic Execution; Boundary Failure; Tool-Using Systems; Action Capability; Constraint Lag; Irreversibility; Permission Overhang; Suggestion–Action Collapse; Human-in-the-Loop; Rollback; Execution Scope; Agent Governance; Responsibility Alignment; Meta-Writing Ecology.

1. Introduction

Interpretive systems can be wrong without immediately changing the external field. Their outputs may still require a person to copy, submit, authorize, or execute an action. Tool-using systems reduce or remove that gap.

The transition is not merely an increase in capability. It changes the location of risk. A misunderstanding can become a sent message, an altered record, a financial transaction, a published statement, or a deleted resource. The cost of error depends not only on model quality but also on permissions, timing, reversibility, confirmation, and the architecture through which action becomes effective.

This model treats execution as a boundary transition. It asks whether the system's constraints have been redesigned for action rather than inherited from a suggestion-only environment.

2. Core Definition

Agentic Execution Boundary Failure Model describes a structural failure in which an interpretive, advisory, generative, or analytic system gains execution

authority without receiving a corresponding upgrade in boundary definition, constraint architecture, verification requirements, and reversibility safeguards.

The model applies when a system that previously produced outputs, suggestions, interpretations, plans, classifications, summaries, or recommendations becomes capable of directly altering an operational environment.

The failure does not originate from intention.

It originates from a mismatch between:

expanded execution capability
and
unchanged interpretive-level constraints

The model isolates the moment when interpretation becomes action without sufficient boundary reinforcement.

3. Classification and Scope

This document presents **Agentic Execution Boundary Failure Model** as an external-facing conceptual model. A model in this context identifies a repeatable structural mechanism, its components, sequence, failure patterns, and limits. It is not an empirical law, a predictive score, a software specification, or a compliance standard.

The model is intended to support description, comparison, and diagnostic inquiry across domains. Application to a specific case requires domain evidence and should not be inferred from conceptual resemblance alone. The public version omits non-public editorial records, private source materials, non-public working materials, and implementation details.

4. Central Distinctions

The model distinguishes:
output generation
from
state alteration

It also distinguishes:
recommendation
from
execution

classification
from
routing

analysis
from
intervention

planning
from
deployment

permission
from
authority

capability
from
governed capability

The presence of execution capacity changes the structural status of the system.

A system that can act is not merely a system that can describe action.

5. Core Mechanism and Structural Sequence

5.1 Core Mechanism

The model operates through **interpretation-action collapse**.

A system begins as non-executing:

interpretive
advisory
generative
analytic
classificatory
planning-oriented

It is then connected to execution capacity:

delete
overwrite
send
approve
reject
route
purchase
deploy
modify
publish
trigger
escalate
assign
terminate
lock
unlock

If the system's boundaries remain calibrated for interpretation rather than execution, it may perform high-impact actions under low-friction conditions.

The failure occurs when the authority to act expands faster than the architecture that constrains action.

5.2 Structural Sequence

1. A system is designed or understood as interpretive, advisory, analytic, or generative.
2. The system produces outputs that appear non-final or reviewable.
3. Execution capability is added through tools, permissions, integrations, workflows, agents, scripts, APIs, routing authority, or operational access.
4. The system can now alter external state.
5. Boundary rules remain implicit, weak, inherited from the advisory phase, or distributed across unclear layers.
6. Verification requirements do not match the cost or irreversibility of action.
7. Reversibility, rollback, audit, escalation, or human authorization remains insufficient.
8. The system performs or triggers a state-changing action.
9. Damage, lock-in, deletion, misrouting, overreach, or responsibility ambiguity occurs.
10. Additional constraints may be added after the failure as residue.

6. Structural Components

The model contains seven primary components:

1. **Execution capability** — the actions the system can perform beyond generating a recommendation.
2. **Permission scope** — the accounts, tools, resources, and environments available to the system.
3. **Verification gate** — the evidence and review required before action becomes effective.
4. **Consequence scale** — the range, speed, persistence, and affected field of the action.
5. **Reversibility** — the degree to which technical and non-technical effects can be restored.
6. **Oversight location** — the node that can inspect, interrupt, confirm, or refuse execution.
7. **Trace and recovery architecture** — the records and mechanisms needed to reconstruct and repair an action.

Boundary failure occurs when execution capability and consequence scale expand faster than these surrounding components.

7. Primary Failure Patterns

1. Suggestion-Action Collapse

system generates recommendation
→ recommendation is treated as executable instruction
→ action occurs without independent authorization

The boundary between output and operation dissolves.

2. Low-Friction High-Impact Execution

minor prompt or local instruction
→ high-impact operation
→ insufficient escalation

The effort required to trigger action is lower than the consequence of action.

3. Verification Gap

- action available
- verification not upgraded
- irreversible operation proceeds

The system verifies as if it were still advisory.

4. Permission Overhang

- system receives broad tool access
- task requires narrow action
- unused capability remains available

The system carries more execution capacity than the task structurally requires.

5. Reversibility Assumption

- system acts
- operators assume correction remains possible
- action proves costly or irreversible

A reversible interface surface conceals irreversible downstream state change.

6. Human-in-the-Loop Misplacement

- human appears present
- human does not control the decisive threshold
- execution remains effectively unbounded

Human presence is mistaken for human control.

7. Post-Failure Constraint Residue

- failure occurs
- new restrictions are added
- residue accumulates around the damaged boundary

The system becomes more constrained after damage, but the original boundary failure remains structurally significant.

8. Operational Evaluation

1. What action can the system execute without a separate human operation?
2. Which permissions make that action possible?
3. Does the system distinguish proposal, preparation, authorization, and execution?

4. What evidence is available to the confirming node before the action becomes effective?
 5. Can the action be reversed technically, institutionally, and socially?
 6. Are high-impact actions subject to independent thresholds or staged execution?
 7. Who is answerable for the action and who can interrupt it?
 8. What traces remain sufficient for reconstruction, correction, and review?
 9. Have constraints been redesigned for action capability, or inherited from a suggestion-only system?
-

9. Application Domains

1. Communication agents

Drafting and sending must remain distinct where recipients, timing, confidentiality, or institutional representation matter.

2. Data and file operations

Modification, deletion, transfer, and publication require permissions, previews, logs, and rollback proportionate to consequence.

3. Financial and procurement workflows

Recommendations should not become transactions without thresholds, authorization, and independent confirmation.

4. Administrative automation

Eligibility, scheduling, routing, and record changes require contestability and clear responsibility for system-initiated actions.

5. Software and infrastructure agents

Code execution, deployment, configuration, and access changes require bounded scopes, staged environments, and recovery mechanisms.

10. Relations to Adjacent Public Frameworks

1. Boundary Failure Diagnostics

Boundary Failure Diagnostics provides a general vocabulary for locating non-entry, closure, lock-in, and boundary loss. The present entry narrows that diagnostic field to a repeatable mechanism.

2. Responsibility Alignment Model

Responsibility Alignment Model clarifies whether ownership, authority, capacity, visibility, and accountability remain attached to compatible nodes. The present entry specifies one condition under which that alignment can fail or become nominal.

3. Verification Labor Compression

Verification Labor Compression describes how accelerated production can move validation work downstream, hide it, or concentrate it at a final review node. The present entry identifies the particular object or boundary that the verification work must preserve.

4. Constraint Residue Accumulation Model

Constraint Residue Accumulation Model describes how unresolved failures leave restrictions, workarounds, and control layers that persist after the initiating event. The present entry identifies one pathway through which such residue can accumulate.

5. Boundary Engineering in High-Stakes Institutions

Boundary Engineering in High-Stakes Institutions treats boundaries as designed conditions for preserving role, authority, evidence, and consequence. The present entry offers a specific boundary that can be examined without turning it into a universal compliance rule.

11. Boundary Conditions and Non-Applicability

11.1 Boundary Conditions

This model applies when:

1. A system has interpretive, advisory, analytic, or generative capacity.
2. The system gains direct or indirect execution capability.
3. Execution can alter external or internal system state.
4. Boundary conditions do not clearly separate output from action.
5. Verification, escalation, rollback, or audit mechanisms do not match action severity.
6. Irreversible or high-cost outcomes are possible.

This model is weak or absent when:

1. The system has no execution capacity.
2. Execution remains fully separated from interpretation.
3. All state-changing action requires independent authorization.
4. Actions are trivially reversible.
5. Constraint architecture is already calibrated to execution risk.
6. The system only produces non-operational text or analysis.

11.2 Non-Applicability

This model should not be reduced to:

AI safety incident
software bug
automation failure
operator error
bad prompt
tool misuse
product defect
cybersecurity breach
workflow inefficiency

These may be application surfaces.

They are not the source ontology of the model.

The model concerns a structural boundary failure produced when an interpretive system is allowed to execute without sufficient constraint transformation.

12. Public Orientation and Method Boundary

This document is designed as a public conceptual entry. It provides a definition, mechanism, diagnostic orientation, and limit statement. It omits non-public editorial records, private source materials, non-public working materials, and implementation details.

The entry is not a substitute for empirical research, legal analysis, professional judgment, technical testing, or domain-specific governance. It should not be used to score people or institutions, automate high-stakes decisions, or infer misconduct from conceptual similarity.

Examples and application domains are illustrative. A case should be classified under **Agentic Execution Boundary Failure Model** only when the relevant structure can be demonstrated from available evidence.

13. Limitations

1. No technical safety specification

The model does not prescribe a particular access-control system, sandbox, or security architecture.

2. Risk varies by action

The same agentic capability can be low-risk in one environment and high-risk in another.

3. Human confirmation is not sufficient by itself

A confirmation step can become ceremonial when the human lacks time or information.

4. Reversibility is graded

Rollback may restore a technical state without reversing social, financial, legal, or reputational consequences.

5. No performance claim

The model does not estimate agent reliability or compare systems.

14. Minimal Formulation

Agentic Execution Boundary Failure occurs when an interpretive system gains execution authority before its constraint architecture is upgraded for state-changing action.

Compressed form:

Interpretation becomes execution before the boundary is rebuilt.

15. Conclusion

Agentic execution boundary failure occurs when action capability expands while the architecture of authority, verification, reversibility, and answerability remains calibrated to interpretation or suggestion. The resulting failure is not reducible to a wrong output. It is a mismatch between what the system can do and the boundary conditions that govern doing.

A structurally aligned agentic system makes execution scope, consequence, confirmation, traceability, rollback, and responsibility explicit. It preserves the distinction between a generated proposal and an externally effective act.

Naming Declaration

Agentic Execution Boundary Failure Model is designated within Meta-Writing Ecology as a structural model describing the failure that occurs when interpretive or generative systems gain execution authority without corresponding boundary recalibration.

The name does not refer to any specific AI model, software platform, company, workflow, organization, incident, or real-world event.

It refers to an abstract structural condition:

interpretive capability
+
execution access
+
insufficient boundary upgrade
=
state-changing failure risk

The model should be used when the central issue is not that a system produced a wrong output, but that the system was positioned to convert output into operation without adequate execution-layer constraints.